Performance & Code Optimization Notes

FAO IDEA Frontend Developer Technical Test – Andrea Giovanni Perozziello

---

1. Summary of the Original Code

The initial component was functional but had several issues:

- API logic was embedded directly in the component
- Multiple `setLoading(false)` calls could cancel each other out
- Duplicate `fetch` blocks meant repeated logic and more maintenance overhead
- No caching or performance optimization was in place
- Error handling was missing

---

2. Key Improvements Made

Code Refactoring

- **Encapsulated API logic** into a reusable service (`api.js`)
- Removed `useEffect` and `useState` in favor of `useQuery` (React Query v5)

State & Data Management

- Integrated **React Query** for:
  - Caching API results
  - Managing loading/error states
  - Handling retries automatically

Performance Optimization

- Used `useMemo` to prevent re-renders when user/project data didn’t change
- Ensured the component re-renders only on actual data change

Error Handling

- Added basic try/catch logic and fallback UI messaging

---

3. Performance Testing Approach

I compared the original and refactored behavior using:

- **Browser DevTools:**
  - Watched for repeated network calls on re-renders
  - Verified that React Query prevented unnecessary fetches
- **React DevTools:**
  - Checked component re-renders and ensured minimal updates
- **Simulated API Delays:**
  - Added random errors and latency to test loading/error states

---

4. Why This Version is Better

Feature	Before	After
API structure	Repeated `fetch` logic	Centralized in `api.js`
State handling	Manual w/ `useState`	Automatic via React Query
Caching	None	Query-based caching
Re-renders	Frequent	Memoized lists
Error handling	Missing	Present
Maintainability	Low	Modular and scalable

---

5. Tools & Libraries Used

- React Query v5 – for caching, async state, and performance
- React hooks (`useMemo`) – to reduce unnecessary rendering
- Mock API – using local data + `setTimeout` to simulate real latency
- ESLint & Prettier – for formatting and consistent code style

---

6. Future Optimizations

- Implement pagination or infinite scroll for large datasets
- Add global error boundary
- Add loading skeletons instead of basic `<p>Loading...</p>`
- Write unit tests for the API layer and UI rendering logic

---

7. Maintainability, Scalability & Code Consistency

Strategies for Maintainability & Scalability

To ensure the codebase stays clean and maintainable as it grows:

- I modularized responsibilities: API logic lives in a separate service (`api.js`), and UI logic stays in the component.
- I used React Query to abstract async state management, which removes the need for manual `useState` / `useEffect` patterns and ensures data access stays declarative and centralized.
- I applied React hooks like `useMemo` to avoid unnecessary re-renders and promote performance efficiency.
- The app is designed so that adding new data sources or views would require minimal change to existing logic — enabling team scaling.

Personal Experience

In previous work on multi-person React projects, I’ve experienced:

- Merge conflicts due to inconsistent formatting
- Code duplication from poor separation of concerns
- Difficulty onboarding new developers without a clear architecture

To solve this, I typically introduce:

- A consistent folder structure (by feature or concern)
- Shared reusable logic (via hooks or services)
- Documentation and walkthroughs for setting up the local environment

This test project follows that same mindset — lean, scalable, and clear.

Code Consistency in Teams

For this test, I implemented the following:

- ESLint with Flat Config for modern linting
- Prettier, fully integrated with eslint-plugin-prettier
- Airbnb style guide as base formatting reference
- React version detection for compatibility and warning suppression

In real-world projects, I’d add Husky + lint-staged to automate linting and formatting on each commit. This helps prevent formatting drift over time, especially in larger teams.

Thanks for reading, and I hope this shows the thinking behind my decisions. I tried to balance simplicity with best practices and clarity.